

Teoria Współbieżności

Laboratorium 10 Część I

Teoria śladów

Kacper Bieniasz

16 grudnia 2024

1 Zadanie 1

Rozważmy zbiór zmiennych („bazę danych”) $\{x, y, z\}$ i następujący zbiór akcji („transakcji”) modyfikujących wartości tych zmiennych:

- (a) $x := x + y$
- (b) $y := y + 2z$
- (c) $x := 3x + z$
- (d) $z := y - z$

Akcje możemy wykonywać współbieżnie z następującym zastrzeżeniem: akcja zmieniająca wartość zmiennej nie może być wykonana współbieżnie z akcją odczytującą lub modyfikującą stan tej samej zmiennej. W języku teorii śladów: dwie akcje są zależne jeśli obie operują na tej samej zmiennej, a przynajmniej jedna z nich modyfikuje wartość tej zmiennej.

1.1 Zadanie 1a

W alfabecie $A = \{a, b, c, d\}$ określ relacje zależności $D(x, y)$ i niezależności $I(x, y)$, gdzie $x, y \in A$.

Bazując na wniosku iż, dwie akcje są zależne jeśli obie operują na tej samej zmiennej, a przynajmniej jedna z nich modyfikuje wartość tej zmiennej. Relacja zależności między operacjami z alfabetu A wygląda następująco:

- $D(a, a)$ - zależne (modyfikujemy x)
- $D(a, b)$ - zależne (modyfikujemy x, y)
- $D(a, c)$ - zależne (modyfikujemy x)
- $D(b, a)$ - zależne (modyfikujemy y)
- $D(b, b)$ - zależne (modyfikujemy y)
- $D(b, d)$ - zależne (modyfikujemy y, z)
- $D(c, a)$ - zależne (modyfikujemy x)
- $D(c, c)$ - zależne (modyfikujemy x)
- $D(c, d)$ - zależne (modyfikujemy z)
- $D(d, b)$ - zależne (modyfikujemy y, z)
- $D(d, c)$ - zależne (modyfikujemy z)
- $D(d, d)$ - zależne (modyfikujemy z)

Zdefiniujmy teraz kiedy dwie akcje są niezależne. Muszą operować na różnych zmiennych, lub jeśli operują na wspólnych to nie mogą ich jednocześnie modyfikować. Relacja niezależności między operacjami z alfabetu A wygląda następująco:

- $I(a, d)$ - niezależne
- $I(d, a)$ - niezależne
- $I(b, c)$ - niezależne
- $I(c, b)$ - niezależne

Dodatkowo wizualnie relację zależności i niezależności możemy przedstawić w formie tabelki. Gdzie D oznacza zależne akcje, a I niezależne. Widzimy, że akcje niezależne znajdują się na przekątnej powstałej tabelki.

	a	b	c	d
a	D	D	D	I
b	D	D	I	D
c	D	I	D	D
d	I	D	D	D

1.2 Zadanie 1b

Wyznacz ślad wyznaczony przez słowo $w = baadcb$ względem powyższej relacji niezależności.

Ślad w teorii śladów to klasa równoważności ciągów akcji (operacji), gdzie dwa ciągi są równoważne, jeśli można je uzyskać z jednego przez przestawianie działań, które są niezależne od siebie. W związku z tym chcemy wyznaczyć wszystkie możliwe ułożenia liter w słowie w . Jednak przestawiać możemy tylko te, które są ze sobą w relacji niezależności I .

Bazując na wnioskach z zadania 1a pierwszą analizowaną parą zdarzeń będzie para a, d . Jak wiemy są niezależne więc mogą następować w dowolnej kolejności tzn. najpierw może być a , po nim d lub odwrotnie. Zmienne, które można przestawiać zapiszę w nawiasach. Słowo w_1 uwzględnia niezależności zdarzeń a i d , wygląda następująco:

$$w_1 = ba(ad)cb$$

Wykonując podobnie działanie dla pary niezależnej b oraz c dochodzimy do słowa w_2 :

$$w_2 = ba(ad)(cb)$$

Ostateczny ślad słowa w wygląda następująco:

$$[w] = \{baadcb, baadbc, badacb, badabc\}$$

1.3 Zadanie 1c

Wyznać postać normalną Foaty śladu $[w]$ można skorzystać z algorytmu z pracy Volker Diekert, Yves Métivier : Partial Commutation and Traces str.11.

Postać normalna Foaty konstruuje słowo ($[w]$) w formie zbioru warstw niezależnych, gdzie każda warstwa zawiera działania, które mogą być wykonane równocześnie. W celu wyznaczenia tej postaci skorzystam z proponowanego algorytmu, którego kroki wyglądają następująco:

1. Pierwszym etapem jest stworzenie stosu dla każdej litery ze słowa w .
2. Następnie przechodzimy do przetwarzania słowa w od prawej do lewej w następujący sposób:
 - Każdą literę a umieszczamy na stosie przypisanym do a .
 - Na stosy wszystkich liter b , które są zależne z a , dodajemy znaczniki " $*$ ".

3. Po przetworzeniu całego słowa w możemy przejść do budowania postaci normalnej Foaty w następujący sposób:

- Grupujemy litery na szczycie każdego ze stosów w porządku leksykograficznym.
- Usuwamy je oraz odpowiadające im znaczniki (tzn. te które razem z nimi zostały umieszczone na stosie) ze stosu.
- Tak wyznaczona grupa liter jest kolejną warstwą w formie normalnej Foaty.
- Powtarzamy grupowanie liter dopóki stosy nie będą puste.

Wykonanie algorytmu na słowie $w = baadcb$:

1. Tworzymy puste stosy odpowiadające każdej literze ze słowa w :

a	b	c	d

2. Skanujemy słowo od prawej do lewej umieszczając litery i znaczniki na stosach.

- Przetwarzamy b

*	b		*
a	b	c	d

- Przetwarzamy c

*			*
*	b	c	*
a	b	c	d

- Przetwarzamy d

			d
*	*	*	*
*	b	c	*
a	b	c	d

- Przetwarzamy a

a	*	*	d
*	*	*	*
*	b	c	*
a	b	c	d

- Przetwarzamy a

a	*	*	
a	*	*	d
*	*	*	*
*	b	c	*
a	b	c	d

- Przetwarzamy b

*	b		
a	*	*	*
a	*	*	d
*	*	*	*
*	b	c	*
a	b	c	d

3. Grupujemy litery na szczytach stosów usuwamy je i znaczniki:

- Pierwsza grupa to (b) . Stos po usunięciu pierwszej warstwy:

a	*	*		
a	*	*	d	
*	*	*	*	
*	b	c	*	
a	b	c	d	

- Następną grupą będzie (ad) . Stos po usunięciu:

a				
*	*	*	*	
*	b	c	*	
a	b	c	d	

- Kolejną grupą będzie (a) . Stos po usunięciu:

*			*	
*	b	c	*	
a	b	c	d	

- Ostatnią grupą będzie (bc) , po usunięciu tych liter oraz znaczników im odpowiadających, każdy stos będzie pusty.

Postać normalna Foata słowa śladu $[w]$ słowa $w = baadcb$ wyznaczona zaproponowanym algorytmem wygląda następująco:

$$Foata([w]) = (b)(ad)(a)(bc)$$

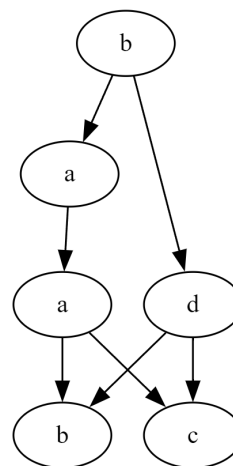
1.4 Zadanie 1d

Narysuj graf zależności Dickerta (w postaci zminimalizowanej - bez krawędzi "przechodnich") dla słowa w .

Graf zależności Dickerta dla słowa w jest skierowanym grafem acyklicznym, który pokazuje zależności między literami w słowie. Aby skonstruować taki graf w zminimalizowanej postaci (bez krawędzi przechodnich) należy:

1. Każdą literę słowa w umieścić jako osobny wierzchołek.
2. Dodać krawędzie między wierzchołkami zależnymi, ale wchodzące tylko z litery będącej wcześniej w słowie w .
3. Jeśli wystąpiły krawędzie przechodnie to je usuwamy.

Postępując według powyższych zasad wyznaczony graf Dickerta dla słowa w przedstawia rysunek (1).



Rysunek 1: Graf zależności Dickerta dla słowa w .

2 Zadanie 2

Dany jest zbiór akcji:

- (a) $x \leftarrow y + z$
- (b) $y \leftarrow x + w + y$
- (c) $x \leftarrow x + y + v$
- (d) $w \leftarrow v + z$
- (e) $v \leftarrow x + v + w$
- (f) $z \leftarrow y + z + v$

2.1 Zadanie 2a

W alfabecie $A = \{a, b, c, d, e, f\}$ określ relacje zależności i niezależności.

Postępując analogicznie jak w zadaniu 1a, wykorzystując wniosek zależności (1), wyznaczmy relację zależność D oraz niezależności I . Zbiór par zależnych prezentuje się następująco:

- | | |
|--|--|
| • $D(a, a)$ - zależne (modyfikujemy x) | • $D(d, b)$ - zależne (modyfikujemy w) |
| • $D(a, b)$ - zależne (modyfikujemy x, y) | • $D(d, d)$ - zależne (modyfikujemy w) |
| • $D(a, c)$ - zależne (modyfikujemy x) | • $D(d, e)$ - zależne (modyfikujemy w, v) |
| • $D(a, e)$ - zależne (modyfikujemy x) | • $D(d, f)$ - zależne (modyfikujemy z) |
| • $D(a, f)$ - zależne (modyfikujemy z) | • $D(e, a)$ - zależne (modyfikujemy x) |
| • $D(b, a)$ - zależne (modyfikujemy x, y) | • $D(e, c)$ - zależne (modyfikujemy x, v) |
| • $D(b, b)$ - zależne (modyfikujemy y) | • $D(e, d)$ - zależne (modyfikujemy w, v) |
| • $D(b, c)$ - zależne (modyfikujemy x, y) | • $D(e, e)$ - zależne (modyfikujemy v) |
| • $D(b, d)$ - zależne (modyfikujemy w) | • $D(e, f)$ - zależne (modyfikujemy v, z) |
| • $D(b, f)$ - zależne (modyfikujemy y) | • $D(f, a)$ - zależne (modyfikujemy z) |
| • $D(c, a)$ - zależne (modyfikujemy x) | • $D(f, b)$ - zależne (modyfikujemy y) |
| • $D(c, b)$ - zależne (modyfikujemy x, y) | • $D(f, d)$ - zależne (modyfikujemy z) |
| • $D(c, c)$ - zależne (modyfikujemy x) | • $D(f, e)$ - zależne (modyfikujemy v) |
| • $D(c, e)$ - zależne (modyfikujemy x, v) | • $D(f, f)$ - zależne (modyfikujemy z) |

Natomiast zbiór zmiennych zależnych I jest znacznie mniejszy:

- | | |
|--------------------------|--------------------------|
| • $I(a, d)$ - niezależne | • $I(d, a)$ - niezależne |
| • $I(b, e)$ - niezależne | • $I(d, c)$ - niezależne |
| • $I(c, d)$ - niezależne | • $I(e, b)$ - niezależne |
| • $I(c, f)$ - niezależne | • $I(f, c)$ - niezależne |

2.2 Zadanie 2b

Wyznacz postać normalną Foaty śladu $[u]$, $u = acdcfbbe$.

W tym celu wykorzystamy algorytm użyty w zadaniu 1c (1.3). Tworzymy stosy odpowiadające każdej literze:

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>

Rozpoczynamy skanowanie słowa od prawej do lewej, pamiętając o umieszczaniu znaczników na stosach zmiennych zależnych od przetwarzanej litery.

- Stosy po przetworzeniu *e* (pozostałe litery do przetworzenia *acdcfbbe*):

*		*	*	<i>e</i>	*
<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>

- Stos po przetworzeniu *b* (pozostałe litery do przetworzenia *acdcfb*):

*		*	*		*
*	<i>b</i>	*	*	<i>e</i>	*
<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>

- Stos po przetworzeniu *b* (pozostałe litery do przetworzenia *acdcf*):

*		*	*		*
*	<i>b</i>	*	*		*
*	<i>b</i>	*	*	<i>e</i>	*
<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>

- Stos po przetworzeniu *f* (pozostałe litery do przetworzenia *acdc*):

*			*		<i>f</i>
*	*	*	*		*
*	<i>b</i>	*	*	*	*
*	<i>b</i>	*	*	<i>e</i>	*
<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>

- Stos po przetworzeniu *c* (pozostałe litery do przetworzenia *acd*):

*					
*	*	<i>c</i>	*		<i>f</i>
*	*	*	*	*	*
*	<i>b</i>	*	*	*	*
*	<i>b</i>	*	*	<i>e</i>	*
<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>

- Stos po przetworzeniu *d* (pozostałe litery do przetworzenia *ac*):

*	*		<i>d</i>		*
*	*	<i>c</i>	*	*	<i>f</i>
*	*	*	*	*	*
*	<i>b</i>	*	*	*	*
*	<i>b</i>	*	*	<i>e</i>	*
<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>

- Stos po przetworzeniu c (pozostałe litery do przetworzenia a):

*	*				
*	*	c	d	*	*
*	*	c	*	*	f
*	*	*	*	*	*
*	b	*	*	*	*
*	b	*	*	e	*
a	b	c	d	e	f

- Stos po przetworzeniu ostatniej litery a :

a	*				
*	*	*		*	*
*	*	c	d	*	*
*	*	c	*	*	f
*	*	*	*	*	*
*	b	*	*	*	*
*	b	*	*	e	*
a	b	c	d	e	f

Możemy przejść do budowania postaci normalnej Foaty dla śladu $[u]$, zdejmując ze stosów litery oraz dopowiadające im znaczniki grupując tak zdjęte litery w warstwy.

- Pierwszą warstwą będzie (ad) . Stos po usunięciu odpowiednich symboli:

*					
*	*	c			
*	*	c	*	*	f
*	*	*	*	*	*
*	b	*	*	*	*
*	b	*	*	e	*
a	b	c	d	e	f

- Drugą warstwą będzie (cf) . Stos po usunięciu odpowiednich symboli:

*		c			
*	*	*	*		*
*	b	*	*	*	*
*	b	*	*	e	*
a	b	c	d	e	f

- Kolejną warstwą będzie (c) . Stos po usunięciu odpowiednich symboli:

*		*	*		*
*	b	*	*		*
*	b	*	*	e	*
a	b	c	d	e	f

- Kolejną warstwą będzie (be) . Stos po usunięciu odpowiednich symboli:

*	b	*	*		*
a	b	c	d	e	f

- Ostatnią warstwą będzie (b) . Po usunięciu b i odpowiadającym jej znacznikom, każdy stos będzie pusty.

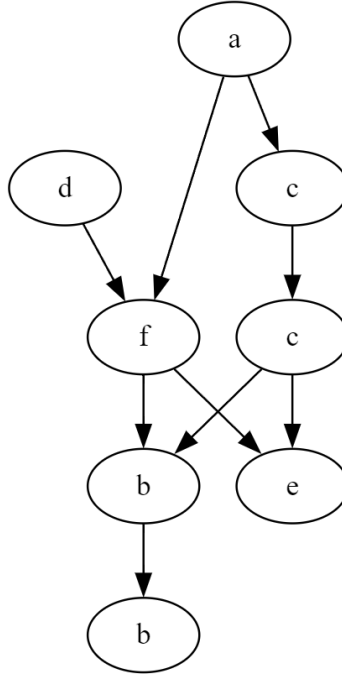
Postać normalna Foaty śladu $[u]$, $u = acdcfbbe$ wyznaczona zaproponowanym algorytmem wygląda następująco:

$$Foata([u]) = (ad)(cf)(c)(be)(b)$$

2.3 Zadanie 2c

Narysuj graf zależności Dickerta (w postaci zminimalizowanej - bez krawędzi "przechodnich") dla słowa u .

Postępując analogicznie według zasad z zadania 1d (1.4). Graf zależności Dickerta dla słowa u prezentuje wykres (2).



Rysunek 2: Graf zależności Dickerta dla słowa u

3 Wnioski

Teoria śladów oraz związane z nią pojęcia, takie jak relacje zależności i niezależności, postać normalna Foaty oraz graf zależności Dickerta, stanowią istotne narzędzia w modelowaniu współbieżności i analizie procesów równoległych. Relacje zależności pozwalają formalnie określić, które akcje mogą być wykonywane jednocześnie, a które wymagają zachowania ustalonej kolejności ze względu na wspólne zasoby. Postać normalna Foaty umożliwia uporządkowanie operacji w warstwy działań równoczesnych, co jest kluczowe dla efektywnej organizacji i optymalizacji procesów współbieżnych. Z kolei graf zależności Dickerta pozwala na wizualizację i analizę tych relacji w sposób przejrzysty, eliminując nadmiarowe krawędzie przechodnie, co upraszcza interpretację zależności w systemie.

Literatura

- [1] <https://www.researchgate.net/publication/280851316-Partial-Commutation-and-Traces>
- [2] https://www.mimuw.edu.pl/~sl/teaching/22_23/TW/
- [3] <https://www.researchgate.net/publication/272946925-Fast-GPU-integration-algorithm-for-isogeometric-finite-element-method-solvers-using-task-dependency-graphs>